

Research Implementation Audit

Project Title: Robust Federated Learning-Based Intrusion Detection for Heterogeneous Vehicular CAN Networks

Target Venue: Research Paper / Academic Publication

Audit Type: Independent Scientific & Methodological Codebase Audit

Date of Audit: October 2026

Auditor: Independent AI Research Systems Auditor (DeepMind Antigravity)

1. Executive Summary

This audit independently inspects the centralized baseline implementation of the research project "*Robust Federated Learning-Based Intrusion Detection for Heterogeneous Vehicular CAN Networks*". The stated goal of the current codebase is to provide a scientifically defensible, leak-free, reproducible centralized 1D-CNN baseline for vehicular CAN bus intrusion detection before extending the framework to decentralized Federated Learning (FedAvg, FedProx, Byzantine-robust aggregation, and Non-IID Dirichlet client partitions).

High-Level Assessment

- 1. Reproducibility:** The centralized training and evaluation pipeline is **100% bit-for-bit deterministically reproducible** on CPU when trained from the preprocessed data (`can_ids_processed_dataset.npz`) with random seed 42. Every reported metric (Accuracy: 99.62%, Precision: 99.72%, Recall: 99.27%, F1-score: 99.50%, FPR: 0.17%, FNR: 0.73%, ROC-AUC: 0.9995) was verified independently to float precision.
- 2. Data Ingestion Flaw (Critical):** A severe CSV parsing bug exists in `src/data/loader.py` (`load_csv_dataset`). The function hardcodes a 12-column schema without accounting for CAN frames with Data Length Code (DLC) < 8 (e.g. DLC = 5 or 2). Consequently, in files such as `Fuzzy_dataset.csv`, 5,439 frames (5.44% of ingested records) suffer from column misalignment: payload bytes are corrupted, the `Flag` column becomes `NaN`, and byte columns are populated with string labels.
- 3. Data Subsampling Scope:** While the raw HCRL dataset contains **17,558,346 CAN messages** across 5 files, the current pipeline subsamples only the first **100,000 frames per scenario** (totaling 500,000 raw frames, or $< 2.85\%$ of available data). This limitation must be explicitly qualified in any paper submission.
- 4. Data Leakage & Splitting:** The chronological 70% / 15% / 15% train/val/test splitting is performed *per scenario* after sliding-window construction. Because window stride equals window length ($S = W = 16$), **no windows cross partition boundaries**, and 0 cross-split duplicate sequences were found. However, a minor technical leak exists in `compute_inter_arrival_times`, where `np.median(diffs)` of the entire 100,000-frame sequence is assigned to frame index 0.
- 5. Latency Reporting Misrepresentation:** The reported inference latency of **28.03 μ s per CAN window** is an amortized batch figure (total batch time of 3.59 ms divided by 128 samples). Independent single-sample testing (`batch_size = 1`) reveals an actual inference latency of **991.96 μ s (~0.99 ms)**, which is 35 times slower. In real-time edge vehicular systems, reporting amortized batch throughput as per-sample detection latency is scientifically misleading.

6. **Federated Learning Suitability:** The lightweight 1D-CNN (~40.6k parameters, 162.4 KB state dict) is structurally well-suited for edge vehicular clients. However, the presence of three `BatchNorm1d` layers presents severe convergence and weight-drift hazards under Non-IID client distributions.

2. Current Project Inventory

The workspace was inspected directly from the filesystem. The table below lists all primary artifacts:

Path / Component	Type	Size / Lines	Status / Purpose
<code>README.md</code>	Documentation	7.2 KB / 141 lines	High-level architecture, metrics and roadmap. Contains discrepancies with actual dataset counts.
<code>requirements.txt</code>	Dependency Spec	1.4 KB / 25 lines	Specifies PyTorch, Pandas, Scikit-Learn, Seaborn, Jupyter. Functional.
<code>01_Dataset_EDA.ipynb</code>	Jupyter Notebook	51.6 KB / 51 cells	Orphaned / Unexecuted draft EDA notebook at root. 0 executed outputs.
<code>notebooks/01_dataset_analysis.ipynb</code>	Jupyter Notebook	241.9 KB / 9 cells	Primary executed EDA notebook. Contains data discovery, class plots, and CAN ID distributions.
<code>notebooks/02_preprocessing.ipynb</code>	Jupyter Notebook	10.8 KB / 5 cells	Feature extraction, sliding window creation, and chronological splitting. Executed.
<code>notebooks/03_baseline_ids.ipynb</code>	Jupyter Notebook	152.1 KB / 10 cells	1D-CNN baseline model training, evaluation, latency profiling, and artifact export. Executed.
<code>src/data/loader.py</code>	Python Module	5.9 KB / 189 lines	Implements <code>load_csv_dataset</code> and <code>load_txt_normal</code> . Contains critical DLC parsing bug.
<code>src/preprocessing/parser.py</code>	Python Module	3.2 KB / 107 lines	Hex ID conversion, payload byte parsing, and log-scaled Δt calculation.
<code>src/preprocessing/pipeline.py</code>	Python Module	9.7 KB / 278 lines	<code>CANPreprocessor</code> , <code>create_sliding_windows</code> , <code>chronological_split</code> , and <code>process_and_split_scenarios</code> .
<code>src/models/cnn1d.py</code>	Python Module	3.4 KB / 121 lines	PyTorch implementation of <code>CAN1DCNN</code> (3 Conv1D blocks, BatchNorm, Linear head).
<code>src/evaluation/metrics.py</code>	Python Module	6.5 KB / 214 lines	Metric computation (Accuracy, Precision, Recall, F1, FPR, FN).

Path / Component	Type	Size / Lines	Status / Purpose
			ROC-AUC), latency profiler, plotting utils.
data/raw/DoS_dataset.csv	Raw Dataset	181.25 MB / 3,665,771 lines	Kia Soul DoS attack capture log.
data/raw/Fuzzy_dataset.csv	Raw Dataset	189.32 MB / 3,838,860 lines	Kia Soul Fuzzy injection attack log.
data/raw/gear_dataset.csv	Raw Dataset	219.65 MB / 4,443,142 lines	Kia Soul Gear spoofing attack log.
data/raw/RPM_dataset.csv	Raw Dataset	228.48 MB / 4,621,702 lines	Kia Soul Engine RPM spoofing attack log.
data/raw/normal_run_data.txt	Raw Dataset	83.32 MB / 988,871 lines	Kia Soul Ambient Normal traffic log.
data/raw/normal_run_data.7z	Archive	7.16 MB	Compressed archive of <code>normal_run_data.txt</code> .
data/processed/can_ids_processed_dataset.npz	Binary Artifact	3.09 MB	Compressed NumPy archive containing <code>X_train</code> , <code>y_train</code> , <code>X_val</code> , <code>y_val</code> , <code>X_test</code> , <code>y_test</code> .
data/processed/preprocessing_metadata.json	Metadata JSON	1.7 KB / 79 lines	Metadata of the processed dataset.
data/splits/	Directory	0 KB / 0 files	Empty directory . Notebook claimed splits would be saved here; nothing was written.
results/models/baseline_1d_cnn.pt	Model Checkpoint	168.7 KB	Saved PyTorch state_dict, training history, and test metrics.
results/metrics/baseline_results.json	Metrics Report	1.0 KB / 44 lines	Standardized JSON results report.
results/figures/	Plot Directory	5 PNG images	Confusion matrix, training curve, class balance, CAN ID distributions, delta-t distribution.

3. Dataset Audit

Raw Dataset Inventory & Statistics

The raw dataset files provided under `data/raw/` were verified through independent streaming line counts and format inspections:

File Name	File Type	Actual Size	Total Records	Header Present?	Subsampled Records Used in Pipeline	Ingestion Ratio
<code>DoS_dataset.csv</code>	Comma-delimited	181.25 MB	3,665,771	No	100,000	2.73%
<code>Fuzzy_dataset.csv</code>	Comma-delimited	189.32 MB	3,838,860	No	100,000	2.60%
<code>gear_dataset.csv</code>	Comma-delimited	219.65 MB	4,443,142	No	100,000	2.25%
<code>RPM_dataset.csv</code>	Comma-delimited	228.48 MB	4,621,702	No	100,000	2.16%
<code>normal_run_data.txt</code>	Space-delimited log	83.32 MB	988,871	No	100,000	10.11%
Total	—	902.02 MB	17,558,346	—	500,000	2.85%

Verified Field Schemas & Semantic Meanings

1. CSV Attack Logs (`DoS`, `Fuzzy`, `Gear`, `RPM`):
 - Format: `<Timestamp>,<CAN_ID>,<DLC>,<DATA[0]>,...,<DATA[DLC-1]>,<Flag>`
 - `Flag`: 'R' indicates regular ambient background frame; 'T' indicates targeted injected attack frame.
2. Normal Run TXT (`normal_run_data.txt`):
 - Format: `Timestamp: <ts> ID: <hex_id> 000 DLC: <dlc> <b0> <b1> ...`
 - Parsed via regex in `src/data/loader.py`: Assigns `Flag = 'R'` and `Attack_Type = 'Normal'`.

Detailed Column Distributions (First 100,000 Frames Ingested)

Scenario	Total Frames	Normal Frames (R)	Attack Frames (T)	Attack Proportion (%)	Unique CAN IDs	Time Duration (s)
DoS	100,000	76,327	23,673	23.67%	27	78.43
Fuzzy	100,000	87,979	12,021	12.02%	1,957	108.68
Gear	100,000	81,183	18,817	18.82%	26	50.78
RPM	100,000	81,048	18,952	18.95%	26	50.31
Normal	100,000	100,000	0	0.00%	27	51.52
Combined	500,000	426,537	73,463	14.69%	—	—

Discrepancies and Ingestion Bugs

1. CRITICAL: Variable DLC Parsing Defect in load_csv_dataset

- **Location:** `src/data/loader.py`, line 91:

```
df = pd.read_csv(
    path,
    header=None,
    names=CAN_CSV_COLUMNS,
    nrows=nrows,
    dtype=str,
    low_memory=False,
)
```

Where `CAN_CSV_COLUMNS` expects exactly 12 columns: `["Timestamp", "CAN_ID", "DLC", "DATA[0]", ..., "DATA[7]", "Flag"]`.

- **Defect:** In standard CAN bus traffic, messages can carry fewer than 8 data bytes. For instance, CAN ID `0x02B0` carries `DLC = 5` (5 data bytes), and CAN ID `0x05F0` carries `DLC = 2` (2 data bytes).
- **Observed In Raw Data:**
 - `Fuzzy_dataset.csv`, line 2: `1478195721.905736,02b0,5,ff,7f,00,05,49,R` (9 fields total).
 - `DoS_dataset.csv`, line 37: `1478198376.409484,05f0,2,01,00,R` (6 fields total).
- **Consequence:** Because `pd.read_csv` matches column names from left to right:
 1. For a `DLC = 5` row, `DATA[5]` receives the string `'R'`, while `DATA[6]`, `DATA[7]`, and `Flag` are assigned `NaN`.
 2. For a `DLC = 2` row, `DATA[2]` receives `'R'`, while `DATA[3]..DATA[7]` and `Flag` are assigned `NaN`.
 3. In `Fuzzy_dataset.csv` alone, **5,439 rows** in the first 100,000 frames have fewer than 11 commas.
 4. In `src/preprocessing/parser.py`, `_to_byte` attempts to parse `'R'` as hex, encounters a `ValueError`, and falls back to `0`. Consequently, **payload bytes are corrupted and shifted for all variable DLC frames**.
 5. In `df["Flag"] == "T"`, since `Flag` is `NaN`, these rows evaluate to `False`. Fortunately, all 5,439 short rows in the sample happened to be regular frames (`Flag = 'R'`); had any injected attack carried `DLC < 8`, it would have been silently misclassified as Normal.

4. Preprocessing Audit

The feature engineering pipeline extracts 11 features per CAN message:

Feature Transformation Equations & Properties

#	Feature Name	Source	Mathematical Transformation	Range	Normalization Type	Defensibility
1	<code>can_id_norm</code>	<code>CAN_ID</code> (hex)	$\text{clip}\left(\frac{\text{int}(\text{CAN_ID}_{16})}{2047.0}, 0.0, 1.0\right)$	<code>[0.0, 1.0]</code>	Min-Max (Domain)	Defensible for 11-bit standard CAN (0x000 – 0x7FF).
2	<code>dlc_norm</code>	<code>DLC</code> (int)	$\text{clip}\left(\frac{\text{DLC}}{8.0}, 0.0, 1.0\right)$	<code>[0.0, 1.0]</code>	Min-Max (Domain)	Defensible. Standard CAN

#	Feature Name	Source	Mathematical Transformation	Range	Normalization Type	Defensibility
						frame length is 0 – 8.
3-10	d0_norm ... d7_norm	DATA[0..7] (hex)	$\frac{\text{int}(D_i, 16)}{255.0}$	[0.0, 1.0]	Min-Max (Domain)	Defensible per byte (0x00 – 0xFF), but corrupted by variable DLC parsing.
11	delta_t_log	Timestamp (float)	$\log_{10}(1.0 + \Delta t \times 1000.0)$	[0.0, ~3.0]	Log1p	Questionable scale mismatch; unclipped upper tail; uses sequence median for index 0.

Observations on Feature Transformations

- Scale Disparity:** Features 1 through 10 are strictly bounded in [0.0, 1.0]. In contrast, `delta_t_log` is unnormalized and ranges from 0.0 to $\log_{10}(1001) \approx 3.0004$ (given the 1.0s clipping threshold). While `BatchNorm1d` mitigates scale differences during training, passing heterogeneous scale features into standard distance-based aggregation algorithms in subsequent Federated Learning (e.g., Krum, coordinate-wise median) will disproportionately weight Δt .
- Missing Value Handling:**
 - Missing/invalid CAN IDs default to 0.
 - Missing DLC values default to 8.
 - Missing payload bytes default to 0.
 - Timestamps with non-monotonic jumps or negative diffs are clipped at `a_min = 0.0` and `a_max = 1.0`.
- Delta-t Index Zero Imputation:** In `src/preprocessing/parser.py`, line 100: `deltas[0] = np.median(diffs) if len(diffs) > 0 else 0.001` This computes the median of all 100,000 message inter-arrival times across the entire scenario file and assigns it to the first message.

5. Data Leakage Audit

Verdict: WARNING (Minor technical leak in sequence median computation; structural split is leak-free).

An exhaustive investigation was conducted to address the 10 data leakage criteria:

Audit Question	Verified Finding	Leakage Status	Evidence / Code Trace
1. Split Location	Split occurs in <code>process_and_split_scenarios</code> after feature conversion and window construction.	PASS	<code>src/preprocessing/pipeline.py#L226-L240</code>
2. Split vs Normalization	Normalization uses fixed domain constants (2047, 8, 255).	PASS	No training dataset statistics (μ, σ) are computed across partitions.
3. Normalization Statistics Source	Constants are derived from the CAN 2.0 protocol specification, not sample distributions.	PASS	Protocol constants are invariant across train and test sets.
4. Windowing vs Splitting	Windowing occurs before chronological splitting.	PASS (Conditional)	Safe because window stride $S = W = 16$.
5. Multi-region Windows	Windows are split along discrete integer indices: $X[: \text{train_end}]$, $X[\text{train_end} : \text{val_end}]$, $X[\text{val_end} :]$.	PASS	Each window is composed strictly of contiguous frames from within its partition.
6. Overlapping Boundary Windows	Non-overlapping stride ($S = 16$). Window boundary exactly matches partition index.	PASS	Frame $N \times 16 - 1$ is in train; frame $N \times 16$ is in val. No frame overlap exists.
7. Future Timestamp Leakage	Consecutive differences $\Delta t_i = t_i - t_{i-1}$ depend only on prior timestamps.	WARNING	<code>deltas[0]</code> uses the median of all 100,000 frames (including future val/test frames). Affects only frame index 0.
8. Sequence Duplication	Tested all 4,690 test windows against all 21,875 train windows for exact byte-level matches.	PASS	0 identical sequences found (0.00% overlap).
9. Scenario Independence	Splitting is executed independently for each of the 5 raw files before aggregation.	PASS	Each scenario contributes exactly 70% train, 15% val, 15% test.
10. Aggregation Timing	Aggregation (<code>np.concatenate</code>) occurs strictly after per-scenario chronological splitting.	PASS	No cross-scenario message mixing prior to partition boundaries.

6. Windowing Audit

Verdict: PASS (Methodology functions as designed; label semantics require explicit documentation).

Sliding Window Mechanics

- **Window Length (W):** 16 consecutive CAN messages.
- **Stride (S):** 16 messages (non-overlapping).
- **Chronological Ordering:** Preserved contiguously.

- **Partial Windows:** Automatically discarded via integer range indexing: `np.arange(0, n_frames - window_size + 1, step_size)`.

Window Labeling Semantics (`label_mode = "any"`)

In `src/preprocessing/pipeline.py`:

```
if label_mode == "any":
    has_attack = np.any(w_labels == 1)
    y_windows[i] = 1 if has_attack else 0
```

An independent density analysis was executed on all attack windows across scenarios:

Attack Scenario	Total Attack Windows	Mean Injected Frames per Window	Windows with Exactly 1 Injected Frame	Windows with ≥ 8 Injected Frames	Windows with 16 Injected Frames
DoS	2,735	8.66 / 16 (54.1%)	6 (0.2%)	2,373 (86.8%)	0 (0.0%)
Fuzzy	2,000	6.01 / 16 (37.6%)	34 (1.7%)	451 (22.6%)	0 (0.0%)
Gear	3,689	5.10 / 16 (31.9%)	18 (0.5%)	140 (3.8%)	0 (0.0%)
RPM	3,717	5.10 / 16 (31.9%)	1 (0.0%)	65 (1.7%)	0 (0.0%)

Methodological Evaluation

1. **Label Contamination:** For Gear and RPM spoofing attacks, an "Attack" window contains on average **only ~5 attack frames and 11 normal ambient frames**. Only 3.8% of Gear attack windows and 1.7% of RPM attack windows contain ≥ 8 injected frames.
2. **Defensibility:** For an intrusion detection system, setting `label_mode = "any"` is defensible because an alert must trigger if any portion of a sequence contains malicious traffic. However, this constitutes a **weakly supervised multi-instance learning problem**, not a pure sequence classification task. Authors must explicitly document that the model learns to identify attack signatures embedded within largely normal background traffic.
3. **Alternative Recommendation:** For fine-grained evaluation, frame-level classification or sliding windows with causal masking should be evaluated alongside window-level detection.

7. Model Audit

Exact Architecture Verification

The architecture in `src/models/cnn1d.py` (CAN1DCNN) was inspected and verified against PyTorch layer definitions:

CAN1DCNN Layer Breakdown:

Layer	Input Shape	Output Shape	Parameters
Transpose (if needed)	(B, 16, 11)	(B, 11, 16)	0
Conv1D (conv1)	(B, 11, 16)	(B, 32, 16)	32 * 11 * 3 = 1,056
(bias=False)			
BatchNorm1d (bn1)	(B, 32, 16)	(B, 32, 16)	32 weights + 32 biases = 64
ReLU + MaxPool1d(2)	(B, 32, 16)	(B, 32, 8)	0
Conv1D (conv2)	(B, 32, 8)	(B, 64, 8)	64 * 32 * 3 = 6,144
(bias=False)			
BatchNorm1d (bn2)	(B, 64, 8)	(B, 64, 8)	64 weights + 64 biases = 128
ReLU + MaxPool1d(2)	(B, 64, 8)	(B, 64, 4)	0
Conv1D (conv3)	(B, 64, 4)	(B, 128, 4)	128 * 64 * 3 = 24,576
(bias=False)			
BatchNorm1d (bn3)	(B, 128, 4)	(B, 128, 4)	128 weights + 128 biases = 256
ReLU + AdaptiveAvgPool1d	(B, 128, 4)	(B, 128, 1)	0
Squeeze	(B, 128, 1)	(B, 128)	0
Dropout (p=0.2)	(B, 128)	(B, 128)	0
Linear (fc1)	(B, 128)	(B, 64)	128 * 64 + 64 = 8,256
ReLU + Dropout (p=0.2)	(B, 64)	(B, 64)	0
Linear (fc2)	(B, 64)	(B, 2)	64 * 2 + 2 = 130
Total Trainable Parameters:			40,610
Non-trainable Buffers (BN Running Mean/Var/Batches):			451
Total Model Parameters:			41,061

- **Reported Parameter Count:** 40,610 parameters.
- **Independently Calculated Parameter Count:** 40,610 trainable parameters. Verified.

8. Training Procedure Audit

Parameter / Procedure	Stated Specification	Actual Implemented Value	Verification Notes
Random Seed	42	42	Set via <code>torch.manual_seed(42)</code> and <code>np.random.seed(42)</code> . Verified deterministic.
Train / Val / Test Partition	70% / 15% / 15%	70% / 15% / 15%	Implemented per scenario file. Verified.
Optimizer	Adam	Adam	<code>torch.optim.Adam(lr=0.001, weight_decay=1e-4)</code> . Verified.
Learning Rate	0.001	0.001	Constant throughout training (no LR scheduler).
Weight Decay	Stated in code	10^{-4}	Standard L2 regularization.
Batch Size	128	128	Maintained across train, val, and test DataLoaders.
Epochs	10	10	Completed without early termination.
Loss Function	Cross-Entropy	<code>nn.CrossEntropyLoss()</code>	Unweighted cross-entropy.
Class Weighting	None	None	No class imbalance weighting applied despite 39.1% / 60.9% attack/normal

Parameter / Procedure	Stated Specification	Actual Implemented Value	Verification Notes
			ratio.
Early Stopping	None	None	Trains for fixed 10 epochs.
Checkpoint Selection	Best validation model?	Last Epoch Model	The saved checkpoint is simply the state after epoch 10; not the best validation epoch checkpoint.
Test Set Isolation	Untouched until evaluation	Strictly Untouched	Test DataLoader is never called inside the training epoch loop.

9. Metrics Audit

Verified Metric Recalculation

The test set evaluation was re-executed directly on the saved checkpoint (`baseline_1d_cnn.pt`) using `X_test` and `y_test` from `can_ids_processed_dataset.npz`. Every classification metric was independently recomputed from raw predicted logits:

Metric	Stated in JSON / README	Independently Recomputed Value	Discrepancy
True Negatives (TN)	2,895	2,895	0
False Positives (FP)	5	5	0
False Negatives (FN)	13	13	0
True Positives (TP)	1,777	1,777	0
Total Test Samples	4,690	4,690	0
Accuracy	99.6162% (99.62%)	99.616205%	0.000000%
Precision	99.7194% (99.72%)	99.719416%	0.000000%
Recall	99.2737% (99.27%)	99.273743%	0.000000%
F1-Score	99.4961% (99.50%)	99.496081%	0.000000%
False Positive Rate (FPR)	0.1724% (0.17%)	0.172414% (5 / 2,900)	0.000000%
False Negative Rate (FNR)	0.7263% (0.73%)	0.726257% (13 / 1,790)	0.000000%
ROC-AUC	0.999523 (0.9995)	0.99952283	0.000000%

Analysis of the 13 False Negatives (Missed Attacks)

- The 13 false negative test samples were traced back to their individual scenario ground truth:
- Fuzzy Attack: 10 samples** (76.9% of all missed attacks). Test indices: 986, 990, 1027, 1059, 1095, 1471, 1489, 1498, 1511, 1558.

- **Gear Spoofing: 2 samples** (15.4% of misses). Test indices: 2560, 2618.
- **DoS Attack: 1 sample** (7.7% of misses). Test index: 159.
- **RPM Spoofing: 0 samples** (0.0% of misses). 100% detected.

Resolution of Stated vs Actual Dataset Count Discrepancy

The prompt noted:

"Train: 21,807 windows (8,333 Attack, 13,474 Normal); Val: 4,670 windows (1,787 Attack, 2,883 Normal); Test: 4,690 windows (1,777 Attack, 2,900 Normal)."

Investigation reveals:

1. **Test Set Arithmetic:** $1,777 + 2,900 = 4,677 \neq 4,690$. The discrepancy of 13 represents the **13 False Negatives**. Previous documentation mistakenly reported True Positives ($TP = 1,777$) as the total attack ground truth ($1,790 = 1,777 + 13$).
2. **Train and Validation Counts:** The numbers 21,807 and 4,670 do not exist in the active codebase. In the actual dataset:
 - **Train Set:** 21,875 windows (8,553 Attack, 13,322 Normal). $8,553 + 13,322 = 21,875$.
 - **Val Set:** 4,685 windows (1,798 Attack, 2,887 Normal). $1,798 + 2,887 = 4,685$.
 - **Test Set:** 4,690 windows (1,790 Attack, 2,900 Normal). $1,790 + 2,900 = 4,690$.
 - **Total Windows:** $21,875 + 4,685 + 4,690 = 31,250$.

10. Reproducibility Audit

Verdict: FULLY REPRODUCIBLE

An end-to-end retraining run of `CAN1DCNN` was executed from scratch on CPU using the exact pipeline logic from `03_baseline_ids.ipynb` with `seed = 42`. The training dynamics and final test metrics matched the saved baseline artifacts with **0.000000% difference**:

Metric	Original Run	Reproduced Retraining Run	Absolute Difference
Epoch 1 Val F1	0.99441	0.99441	0.00000
Epoch 5 Val F1	0.99187	0.99187	0.00000
Epoch 10 Val F1	0.99329	0.99329	0.00000
Test Accuracy	99.616205%	99.616205%	0.000000%
Test Precision	99.719416%	99.719416%	0.000000%
Test Recall	99.273743%	99.273743%	0.000000%
Test F1-Score	99.496081%	99.496081%	0.000000%
Test ROC-AUC	0.999523	0.999523	0.000000%
Confusion Matrix	[[2895, 5], [13, 1777]]		Identical

11. Computational Performance Audit

Reported Figures

- Training time: **38.91 seconds** (10 epochs on CPU).
- Batch inference latency: **3.59 ms** (Batch size = 128).
- Per-sample latency: **28.03 μ s**.

Independent Profiling & Methodological Critique

A dedicated benchmarking script was executed to evaluate single-sample versus batched inference across 500 iterations following 50 warm-up runs on CPU:

Benchmark Scenario	Batch Size	Measured Latency	Stated in Project	Real-World Validity
Batched Inference	128	4.102 ms $\pm$ 0.308 ms	3.588 ms	Valid for offline evaluation or multi-window buffer processing.
Amortized Per-Sample	128	32.05 μ s	28.03 μ s	Misleading if framed as real-time response time.
True Single-Sample Latency	1	991.96 μ s $\pm$ 337.8 μ s	Not Reported	Realistic edge inference cost on embedded ECU processor.
95th Percentile Single-Sample	1	1,473.95 μ s (1.47 ms)	Not Reported	Crucial for safety-critical CAN bus deadlines (< 2 ms).

Scientific Verdict on Latency

Dividing batch latency by 128 and claiming a detection latency of 28.03 μ s fails peer-review standards in vehicular cyber-physical systems. In real automotive deployments, an ECU cannot buffer 128 windows ($128 \times 16 = 2,048$ CAN messages, equivalent to > 2.0 seconds of traffic) before initiating an intrusion check. Real-time inference occurs at `batch_size = 1`, which takes **~0.99 ms**. While 0.99 ms is still acceptable for high-speed CAN (where message intervals are typically 1 – 10 ms), it must be reported accurately.

12. Federated Learning Readiness

Evaluating the current centralized baseline for immediate integration into Federated Learning (FedAvg, FedProx, SCAFFOLD):

Component / Property	Current Status	FL Implication	Action Required
Model Footprint	40,610 parameters (162.4 KB state dict)	Excellent for constrained V2X and in-vehicle bandwidth.	Keep architecture lightweight.
Weight Serialization	Standard PyTorch <code>state_dict</code>	Clean parameter extraction and aggregation via <code>OrderedDict</code> .	Fully compatible.

Component / Property	Current Status	FL Implication	Action Required
Batch Normalization	3 <code>BatchNorm1d</code> layers present	Major hazard for Non-IID client distributions. Local running statistics diverge, degrading global model performance.	Replace <code>BatchNorm1d</code> with <code>GroupNorm(num_groups=4, num_channels=C)</code> or <code>LayerNorm</code> .
Loss & Class Imbalance	Unweighted Cross-Entropy	Non-IID partitions with extreme class skew (e.g. clients with 0 attacks) will destabilize gradients.	Introduce focal loss or weighted cross-entropy per client.
Deterministic Init	Implicit PyTorch init	If clients initialize locally without a synchronized global seed, initial aggregation fails.	Create explicit <code>server_init()</code> broadcasting identical initial weights.
Client Partitioning Logic	Does not exist	Need Dirichlet Non-IID partitioning ($\alpha \in \{0.1, 0.5, 1.0\}$) per vehicle ECU.	Must be implemented.

13. Digital Twin Scope Assessment

The project proposal mentions a "Lightweight Digital Twin validation layer".

- **Current Status:** Not implemented.
- **Critical Warning for Academic Paper:** Reviewers in IEEE TIFS, IEEE TDSC, or vehicular networking transactions will reject or challenge claims of a "Digital Twin" if the implementation is merely a CAN log replay script.
- **Recommended Framing:**
 - **DO NOT** claim a full "Cyber-Physical Digital Twin" unless real vehicle dynamics (powertrain, steering, braking feedback loops) are simulated in Carla, SUMO, or Vector CANoe.
 - **DO** frame this component accurately as a *"Trace-Driven Hardware-in-the-Loop CAN Replay and Closed-Loop Anomaly Verification Testbed"*.

14. Scientific Validity Assessment

Skeptical Peer Review Critique

1. **Why is the 99.5% F1 score potentially inflated?**
 - **Subsampled Time Windows:** Subsampling the first 100,000 frames from each capture file selects contiguous, steady-state injection intervals. Full 17.5M-record evaluations with varied driving cycles will almost certainly exhibit lower recall, particularly on low-frequency stealth attacks.
 - **DoS and RPM Predictability:** In the HCRL dataset, DoS attacks flood CAN ID `0x000` at extreme frequencies (0.3 ms intervals), while normalKia Soul traffic never uses ID `0x000`. Any model observing ID `0x000` or tiny Δt achieves trivial 100% detection on DoS.

- **Fuzzy Attack Challenges:** The model struggled primarily on Fuzzy injection (10 of 13 false negatives). This reveals that subtle random payload mutations without predictable CAN ID patterns are the true vulnerability of the current model.
 - 2. **Label Imprecision:** Because `label_mode = "any"` labels an entire 16-frame sequence as "Attack" even if only 1 frame is injected, the model is trained with noisy supervisory signals where up to 93% of frames in an attack window are normal background frames.
 - 3. **Absence of Baseline Comparisons:** A paper cannot stand on a single 1D-CNN model. Reviewers will mandate comparative baselines against classical detectors (Isolation Forest, SVM, Random Forest) and temporal deep learning models (LSTM, GRU, Transformer).
-

15. Critical Issues

CRITICAL (Must Fix Before Federated Learning):

1. **Fix Variable DLC Parsing in `src/data/loader.py`:** Modify `load_csv_dataset` to handle variable-length CSV records without misaligning payload bytes or shifting `Flag` into `DATA[DLC]`.
2. **Replace BatchNorm1d with GroupNorm in `src/models/cnn1d.py`:** In decentralized federated optimization with Non-IID client data, BatchNorm causes catastrophic divergence during FedAvg aggregation. Replace with `nn.GroupNorm`.
3. **Synchronized Global Server Model Initialization:** Implement an explicit global weight initialization routine to broadcast identical starting parameters across all simulated clients.

HIGH (Should Fix Before Final Paper Experiments):

4. **Correct Latency Reporting:** Report true single-sample inference latency ($991.96 \mu s$) alongside batched amortized throughput ($28.03 \mu s$).
5. **Harmonize Feature Scaling:** Normalize `delta_t_log` into $[0.0, 1.0]$ by scaling with its known domain maximum (e.g. dividing by $\log_{10}(1001) \approx 3.0004$) to prevent scale dominance in Byzantine-robust distance metrics (e.g. Krum, FoolsGold).
6. **Correct Documentation Counts:** Update `README.md` and documentation to reflect the true processed dataset numbers (Train: 21,875; Val: 4,685; Test: 4,690; Attack: 1,790; Normal: 2,900) instead of the erroneous 1,777 TP figure.
7. **Best-Model Checkpoint Selection:** Modify the training loop to track validation F1 and save the checkpoint with the highest validation performance rather than defaulting to the final epoch model.

MEDIUM (Should Improve if Time Permits):

8. **Expand Dataset Ingestion Volume:** Scale beyond the initial 100,000 frames per file to 500,000 or 1,000,000 frames per scenario to capture broader driving profiles.
9. **Remove Redundant Draft Notebook:** Deprecate or remove the orphaned `01_Dataset_EDA.ipynb` at the workspace root to prevent confusion with `notebooks/01_dataset_analysis.ipynb`.
10. **Implement Competitive Baselines:** Implement baseline comparisons (MLP, Random Forest, BiLSTM, and Isolation Forest).

LOW (Cosmetic / Documentation):

11. **Populate `data/splits/` or Update Metadata:** `data/splits/` is empty despite claims in notebook 02 that split arrays are persisted there. Either save individual scenario partition files or remove the unused folder.

16. Recommended Fixes (Prioritized)

```
Priority 1: src/data/loader.py
    Rewrite CSV parser to split lines by comma dynamically, padding payload bytes
    to 8 elements based on the parsed DLC value, ensuring Flag is always column index -1.

Priority 2: src/models/cnn1d.py
    Replace nn.BatchNorm1d(C) with nn.GroupNorm(num_groups=min(4, C), num_channels=C)
    to safeguard Federated Learning aggregation against Non-IID client drift.

Priority 3: src/preprocessing/parser.py
    Normalize delta_t_log into [0, 1] via fixed domain scaling:
    delta_t_norm = delta_t_log / np.log10(1.0 + max_delta * 1000.0)

Priority 4: src/evaluation/metrics.py & results/metrics/baseline_results.json
    Update latency benchmark to explicitly report single_sample_latency_us (batch_size=1)
    alongside batch_latency_ms (batch_size=128).

Priority 5: README.md
    Correct reported test set class counts: Total Test Attack = 1,790 (TP = 1,777, FN = 13).
```

17. What Is Already Complete

1. **Raw Data Acquisition:** All 5 core HCRL Car-Hacking datasets (`DoS`, `Fuzzy`, `Gear`, `RPM`, `Normal`) exist locally and are verified.
2. **Exploratory Data Analysis:** Comprehensive analysis of CAN IDs, arbitration priorities, inter-arrival times, and class balance is complete in `notebooks/01_dataset_analysis.ipynb`.
3. **Chronological Splitting Framework:** Per-scenario chronological 70/15/15 time-series partitioning is implemented and verified leak-free.
4. **Model Architecture Design:** Lightweight 1D-CNN baseline (~40.6k parameters) is fully coded in PyTorch.
5. **Centralized Baseline Training:** Fixed-seed training loop with cross-entropy and Adam optimizer is complete.
6. **Reproduction Fidelity:** Baseline results (99.62% Acc, 99.50% F1) are verified and 100% deterministically reproducible.

18. What Must Be Implemented Next (Roadmap)

To advance this project into a top-tier research paper, follow this strict sequential implementation roadmap:

```
[Phase 1: Baseline Hardening]
    Step 1: Fix CSV parser in src/data/loader.py for variable DLC.
    Step 2: Replace BatchNorm1d with GroupNorm in src/models/cnn1d.py.
    Step 3: Re-run notebooks 02 and 03 to generate bug-free baseline checkpoints and metrics.

[Phase 2: Federated Simulation Harness]
    Step 4: Implement Dirichlet Non-IID client partitioner (src/federated/dataset.py)
    Parameterize heterogeneous client partitions with alpha in {0.1, 0.5, 1.0} across K=10
```

```
vehicular ECUs.
    Step 5: Implement Federated Server and Client Orchestrator (src/federated/server.py, client.py).
    Step 6: Implement standard aggregation baselines: FedAvg, FedProx (proximal mu parameter), and
SCAFFOLD.

[Phase 3: Adversarial & Byzantine Defense Layer]
    Step 7: Implement vehicular attack/poisoning simulators (label flipping, sign flipping, model
poisoning).
    Step 8: Implement Byzantine-robust aggregation rules:
        - Coordinate-wise Median
        - Trimmed Mean
        - Krum / Multi-Krum
        - FoolsGold (cosine similarity penalty)

[Phase 4: Comparative Benchmarks & Evaluation]
    Step 9: Run extensive empirical matrix:
        (IID vs Non-IID) x (FedAvg vs FedProx vs Robust) x (Benign vs Malicious Clients).
    Step 10: Compare against centralized baseline and classical detectors.
    Step 11: Implement CAN trace-driven replay verification testbed (Trace Replay Evaluation).
```

19. Final Verdict

Verdict: READY AFTER CRITICAL FIXES

Justification:

The centralized baseline pipeline is **conceptually sound, structurally well-organized, and 100% deterministically reproducible**. The reported metrics are genuine and accurately calculated from the saved checkpoint.

However, moving immediately to Federated Learning without addressing the **critical CSV parsing bug** (which silently corrupts variable-length CAN frames) and the **BatchNorm architectural hazard** (which causes severe divergence in Non-IID client aggregation) would compromise the validity of all downstream FL experiments. Once Priority 1 (CSV parser fix) and Priority 2 (GroupNorm conversion) are resolved, the project will possess an exceptionally solid foundation for publication-quality Federated Learning research.